

HARDWARE BASED IR REMOTE RELAY CONTROLLED SYSTEM

Embedded Systems | Digital Logic (No Microcontroller) | Analog Signal Conditioning

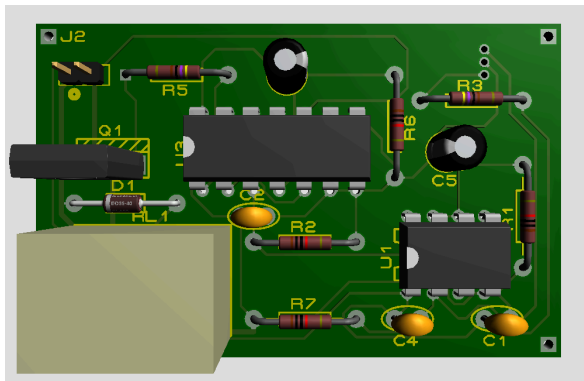
Prepared by: Bouhlel Ahmed

Date: July 2026

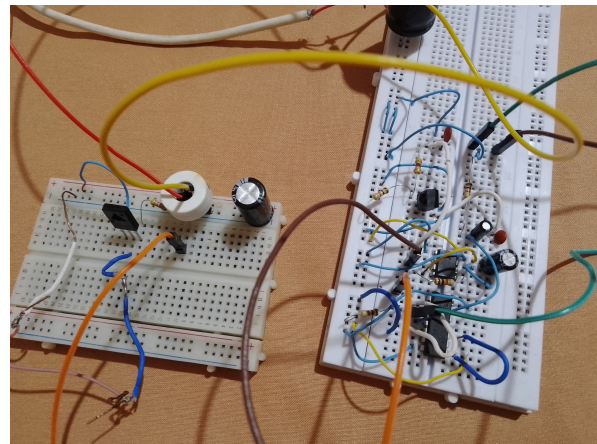
Overview



this project implements an IR remote-controlled relay toggle system using discrete hardware components only. The design processes IR receiver output through signal conditioning, pulse shaping, and flip-flop logic to control a relay-driven load. The system was developed to explore hardware-based digital control .

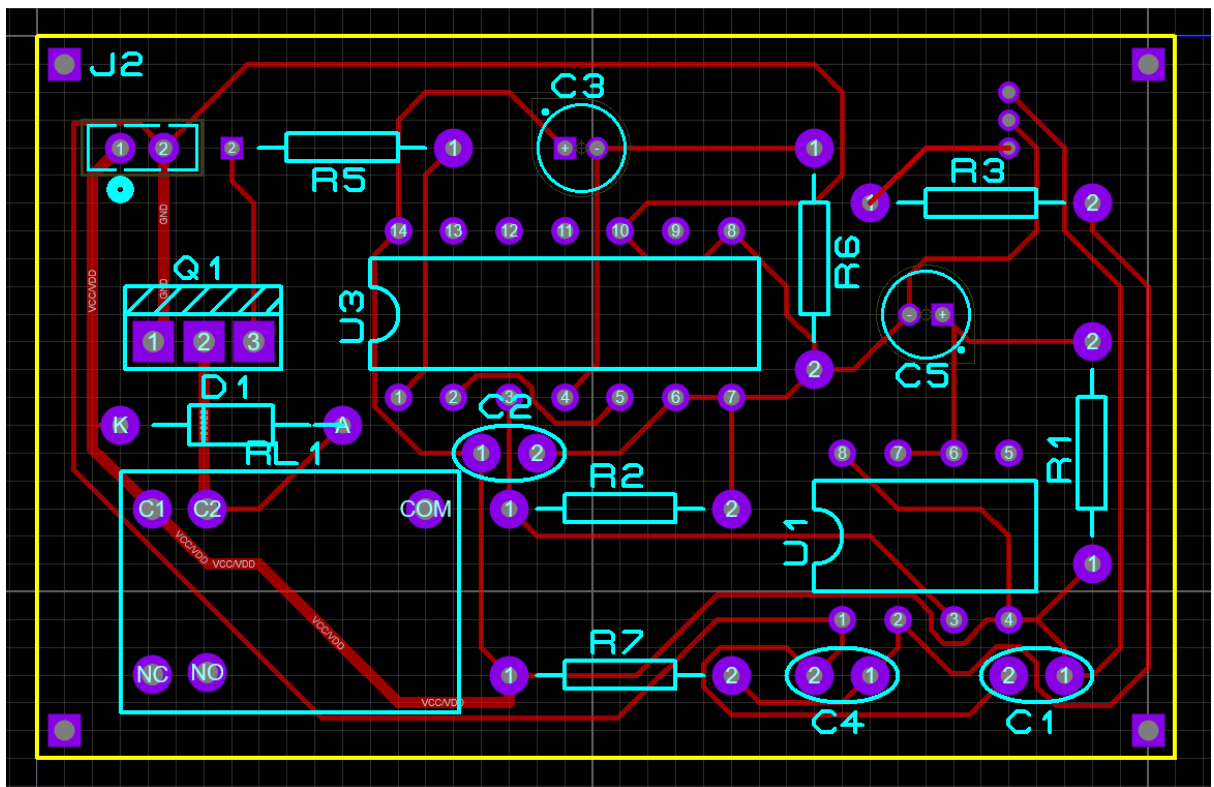
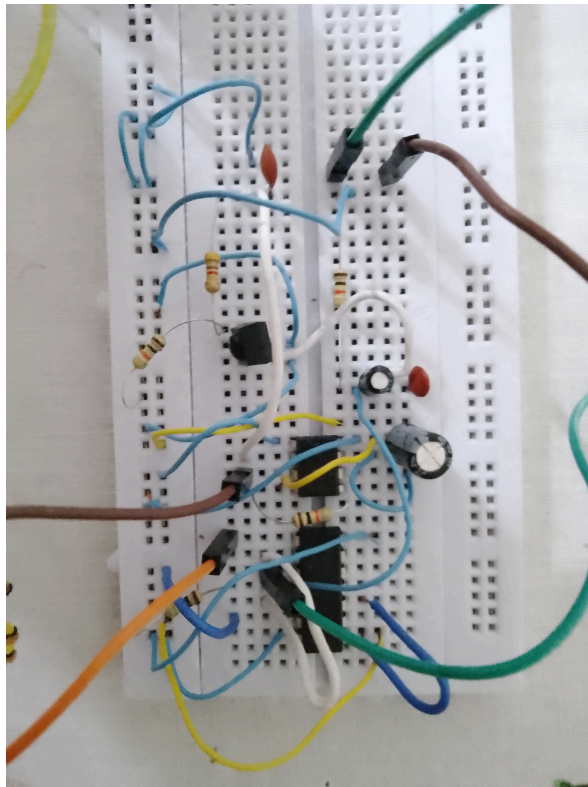


Proteus Ares PCB design



working breadboard version

[click here for video demonstration](#)



Architecture

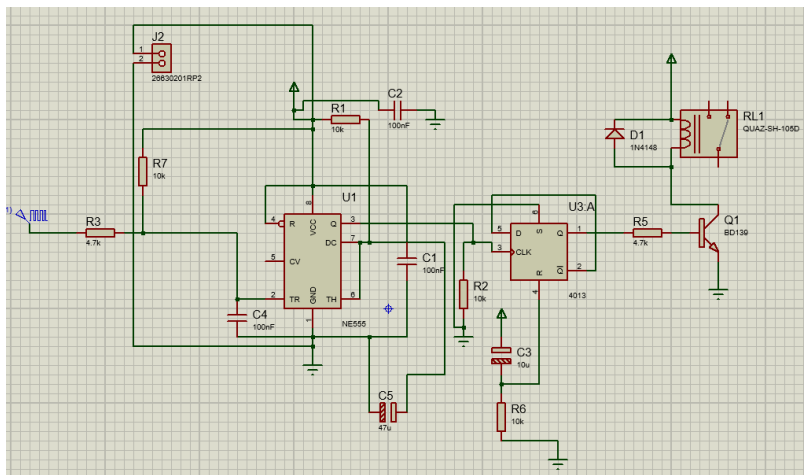


block	Role	details
TSOP IR Receiver ↓	Detects infrared signals from the remote contro	38 kHz IR demodulation
Signal Filtering (RC Low-Pass) ↓	Removes unwanted noise and smooths the signal	added to prevent false triggering caused by noise, which was observed in earlier prototypes.
555 Monostable Pulse Shaper ↓	Generates a fixed-width pulse whenever a valid IR signal is received.	Improved NEC IR protocol signal handling by generating clean trigger pulses
CD4013 Toggle Flip-Flop (D flipflop turned T) ↓	Changes state (ON ↔ OFF) with each pulse from the 555 timer.	added power on reset with an RC circuit to ensure the system starts with “off”
BD139 Transistor Driver ↓	Amplifies the low-current flip-flop output to drive the relay coil	respects cd4013 limiting output(Q) source current
Relay Output Stage	Switches the external load (lamp, appliance, etc.) on or off.	I implemented a lamp as the load in my breadboard version

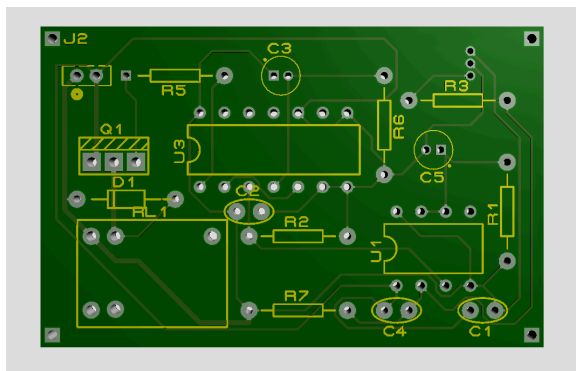
Circuit design



1. I implemented a TSOP IR receiver to enable versatile remote control compatibility, allowing the use of most standard TV remotes. A pull-up resistor and a low-pass RC filter (1 k Ω resistor and 100 nF capacitor) were added to improve signal stability and suppress false triggering observed in early prototypes. Through testing and isolation of faults, I identified the TSOP output stage as the source of noise-induced triggering issues.



2. To further process the signal, a 555 timer in monostable mode was used to generate a clean, fixed-width pulse. The pulse width is defined by the RC time constant ($1.1RC$), set to approximately 500 ms to prevent double triggering while ensuring reliable response timing. Although this introduces a minimum re-trigger delay, it significantly improves system stability.



(proteus isis simulation)

1. For state control, a CD4013 flip-flop was used due to its simplicity, reliability, and low cost, implementing a toggle-based memory function. A power-on reset circuit with a 0.1 s time constant was added to ensure a defined startup state.
2. a BD139 transistor was used as a driver stage to amplify the low-current CMOS output and drive the relay. The flip-flop output sources approximately 0.9 mA, which is sufficient to control the BD139 and energize the relay coil, with a flyback diode included for protection.
3. Finally, the system works on 5 volts, 100 nF caps were added for ics power pins for stability and pullups/pulldowns at CD4013 clock and tsop output.

Debugging & Engineering Challenges



Issue 1: random False triggering

The system initially exhibited unintended switching due to noise on the trigger input. which was later solved with the [low pass filter](#).

Issue 2: ground disturbance by the relay

the relay shifted the ground because it drew a lot of current which messes with the 2 ICs , it was fixed by [decoupling caps](#) and large [reservoir caps](#) and [star ground topology](#).

Issue 3: random power on state

the system started as “on” sometimes and other times as “off” when first connected to power ,which was fixed by [power on reset RC circuit](#)

Results & conclusions



The system reliably toggles a relay using an IR remote input after implementing signal filtering and power stabilization improvements. The final design demonstrates robust hardware-based digital control without microcontroller dependency



[QR code for video demonstration](#)